

```
import math
```

```
EPS1 = 1e-6
```

```
EPS2 = 1e-6
```

```
MAX_ITER = 5000
```

```
# Цільові функції для тестування (Пункт 3)
```

```
def f_rosenbrock(x):
```

```
    return 100.0 * (x[0]**2 - x[1])**2 + (x[0] - 1.0)**2
```

```
def f_power(x):
```

```
    return (10.0 * (x[0] - x[1])**2 + (x[0] - 1.0)**2)**4
```

```
def f_root(x):
```

```
    val = 10.0 * (x[0] - x[1])**2 + (x[0] - 1.0)**2
```

```
    return abs(val)**0.25
```

```
def f_wood(x):
```

```
    return (100.0 * (x[1] - x[0]**2)**2 + (1.0 - x[0])**2 +
```

```
           90.0 * (x[3] - x[2]**2)**2 + (1.0 - x[2])**2 +
```

```
           10.1 * ((x[1] - 1.0)**2 + (x[3] - 1.0)**2) +
```

```
           19.8 * (x[1] - 1.0) * (x[3] - 1.0))
```

```
def f_powell(x):
```

```
    return ((x[0] + 10.0 * x[1])**2 + 5.0 * (x[2] - x[3])**2 +
```

```
           10.0 * (x[0] - x[3])**4 + (x[1] - 2.0 * x[2])**4)
```

```
def f_miele(x):
```

```
    return ((math.exp(x[0]) - x[1])**4 + 100.0 * (x[1] - x[2])**6 +
```

```
           math.tan(x[2] - x[3])**4 + x[0]**8 + (x[3] - 1.0)**2)
```

Цільова функція заданої системи (Пункт 4)

```
def system_func(x):
```

```
    f1 = x[0]**2 + x[1]**2 - 4.0
```

```
    f2 = x[1] - x[0]**2
```

```
    return f1**2 + f2**2
```

Етап 1: Розвідувальний пошук

```
def explore(x_base, dx, func, q, eps1, allow_reduce):
```

```
    n = len(x_base)
```

```
    curr = list(x_base)
```

```
    f_curr = func(curr)
```

```
    for i in range(n):
```

```
        while True:
```

```
            x_forward = list(curr)
```

```
            x_forward[i] += dx[i]
```

```
            if func(x_forward) < f_curr:
```

```
                curr = x_forward
```

```
                f_curr = func(curr)
```

```
                break
```

```
            x_backward = list(curr)
```

```
            x_backward[i] -= dx[i]
```

```
            if func(x_backward) < f_curr:
```

```
                curr = x_backward
```

```
                f_curr = func(curr)
```

```
                break
```

```
    if allow_reduce:
```

```

        dx[i] /= q
        if dx[i] < eps1:
            break
    else:
        break
return curr

```

Етап 2: Пошук за зразком та головний цикл

```
def hooke_jeeves(func, x0, dx_init, q=2.0, p=2.0, eps1=EPS1, eps2=EPS2, filename=None):
```

```
    n = len(x0)
```

```
    x_base = list(x0)
```

```
    dx = list(dx_init)
```

```
    steps = 0
```

```
    file_out = open(filename, "w", encoding="utf-8") if filename else None
```

```
    if file_out:
```

```
        file_out.write(f"Початкова точка: {x0},  $\Phi(x) = \{func(x0):.6e\}$ \n\n")
```

```
    while steps < MAX_ITER:
```

```
        steps += 1
```

```
        dx_current = list(dx)
```

```
        x1 = explore(x_base, dx_current, func, q, eps1, allow_reduce=True)
```

```
        dx = list(dx_current)
```

```
    if file_out:
```

```
        file_out.write(f"Крок {steps}: Точка {[round(val, 6) for val in x1]},  $\Phi(x) = \{func(x1):.6e\}$ \n")
```

```
    if x1 == x_base:
```

```
        break

```

```
norm_dx = math.sqrt(sum(d**2 for d in dx))
```

```
if norm_dx < eps1 and abs(func(x1) - func(x_base)) < eps2:
```

```
    x_base = list(x1)
```

```
    break
```

```
while True:
```

```
    x_p = [x1[i] + p * (x1[i] - x_base[i]) for i in range(n)]
```

```
    dx_pattern = list(dx)
```

```
    x2 = explore(x_p, dx_pattern, func, q, eps1, allow_reduce=False)
```

```
    if func(x2) < func(x1):
```

```
        x_base = list(x1)
```

```
        x1 = list(x2)
```

```
    else:
```

```
        x_base = list(x1)
```

```
        break
```

```
if file_out:
```

```
    file_out.write(f"\nПошук завершено. Кількість кроків: {steps}\n")
```

```
    file_out.write(f"Точка мінімуму  $x^*$  = {[round(val, 6) for val in x_base]}\n")
```

```
    file_out.write(f"Мінімум цільової функції  $\Phi(x^*)$  = {func(x_base):.2e}\n")
```

```
    file_out.close()
```

```
return x_base, func(x_base), steps
```

```
def main():
```

```
    q, p = 2.0, 2.0
```

```
    print("ПУНКТ 3: Тестування на цільових функціях")
```

```
    test_functions = [
```

```

("1. Розенброка", f_rosenbrock, [-1.2, 0.0], [0.5, 0.5]),
("2. Степенева", f_power, [-1.2, 0.0], [0.5, 0.5]),
("3. Коренева", f_root, [-1.2, 0.0], [0.5, 0.5]),
("4. Вуда", f_wood, [-3.0, -1.0, -3.0, -1.0], [0.5, 0.5, 0.5, 0.5]),
("5. Пауелла", f_powell, [-3.0, -1.0, 0.0, 1.0], [0.5, 0.5, 0.5, 0.5]),
("6. Мієле", f_miele, [1.0, 2.0, 2.0, 2.0], [0.5, 0.5, 0.5, 0.5])
]

```

```

for name, func, x0, dx in test_functions:

```

```

    x_min, f_min, steps = hooke_jeeves(func, x0, dx)

```

```

    print(f"{name}:  $x^* = \{[round(val, 4) \text{ for } val \text{ in } x\_min]\}$ ,  $\Phi(x^*) = \{f\_min:.2e\}$ , Кроків: {steps}")

```

```

print("\nПУНКТИ 4-5: Розв'язок системи нелінійних рівнянь")

```

```

x0_sys = [1.0, 1.0]

```

```

dx_sys = [0.5, 0.5]

```

```

res_sys, f_sys, steps_sys = hooke_jeeves(system_func, x0_sys, dx_sys, q, p, EPS1, EPS2,
"trajectory.txt")

```

```

print(f"Знайдений корінь системи: {[round(val, 6) for val in res_sys]}")

```

```

print(f"Значення нев'язки  $\Phi(x^*) = \{f\_sys:.2e\}$  (Кроків: {steps_sys})")

```

```

if __name__ == "__main__":

```

```

    main()

```